DBMS PROJECT • REVIEW 2

TOURISM PACKAGE AND TRAVEL BOOKING MANAGEMENT SYSTEM



Jashwanth Sappidi
25WU0102108

Review 2 Objectives

From normalized design to an initial working prototype

01 **Normalize to Third Normal Form (3NF)**

02 **Identify functional dependencies**

03 **Prepare a structured data dictionary**

04 **Create DDL with integrity constraints**

05 **Insert representative sample data**

06 **Write queries, subqueries, aggregates & views**

MAIN MODULES Destination · Package · Itinerary · Departure · Hotel · Transport · Guide · Customer · Booking · Traveller · Payment · Cancellation · Refund

Database Normalization up to 3NF

A disciplined progression that removes repeating, partial and transitive dependencies.

1NF

Atomic structure

Atomic values · unique identifier · no repeating groups

2NF

Full-key dependency

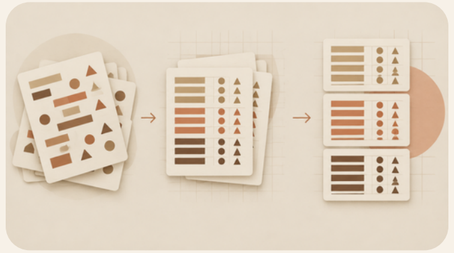
Satisfies 1NF · no partial dependency on candidate keys

3NF

No transitive dependency

Satisfies 2NF · non-key attributes depend only on the key

RESULT Reduced redundancy · stronger consistency · simpler maintenance



Normalized Relational Schema

PK = primary key · FK = foreign key · supporting tables complete hotel and transport mappings

Destination	destination_id PK name, state, country, description
Package	package_id PK destination_id FK, package_name, duration_days, price
Itinerary_Day	itinerary_id PK package_id FK, day_number, activities
Guide	guide_id PK guide_name, phone, specialization
Departure	departure_id PK package_id FK, guide_id FK, dates, capacity, status
Customer	customer_id PK name, email, phone
Booking	booking_id PK customer_id FK, departure_id FK, date, status
Traveller	traveller_id PK booking_id FK, traveller_name, age
Payment	payment_id PK booking_id FK, amount, date, status
Cancellation	cancellation_id PK booking_id FK, date, reason
Refund	refund_id PK cancellation_id FK, payment_id FK, amount, status

Supporting: Hotel · Transport · Package_Hotel · Package_Transport · Departure_Transport

Functional Dependencies

A determinant uniquely establishes the dependent attributes — the basis for candidate keys and normalization.

- customer_id → name, email, phone
- destination_id → name, state, country, description
- package_id → destination_id, package_name, duration_days, price
- itinerary_id → package_id, day_number, activities
- guide_id → guide_name, phone, specialization
- departure_id → package_id, guide_id, dates, capacity, status
- booking_id → customer_id, departure_id, booking_date, status
- traveller_id → booking_id, traveller_name, age
- payment_id → booking_id, amount, payment_date, status
- cancellation_id → booking_id, cancellation_date, reason
- refund_id → cancellation_id, payment_id, amount, refund_date, status

WHY IT MATTERS Identifies candidate keys · reduces duplication · improves integrity

SQL DDL & Integrity Constraints

Declarative constraints protect quality at the database boundary.

```
CREATE TABLE Package (
  package_id INT PRIMARY KEY,
  destination_id INT NOT NULL,
  package_name VARCHAR(100) NOT NULL,
  duration_days INT CHECK (duration_days > 0),
  price DECIMAL(10,2) CHECK (price >= 0),
  FOREIGN KEY (destination_id)
    REFERENCES Destination(destination_id)
);
```

```
CREATE TABLE Departure (
  departure_id INT PRIMARY KEY,
  package_id INT NOT NULL,
  guide_id INT,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  capacity INT CHECK (capacity > 0),
  status VARCHAR(20) DEFAULT 'PLANNED',
  FOREIGN KEY (package_id) REFERENCES
Package(package_id),
  FOREIGN KEY (guide_id) REFERENCES
Guide(guide_id)
);
```



CONSTRAINT DEFINITIONS

PRIMARY KEY: Uniquely identifies each record in a table. It cannot contain duplicate or NULL values.
FOREIGN KEY: Connects two tables and maintains valid relationships between them.
NOT NULL: Ensures that a column cannot have an empty (NULL) value.

UNIQUE: Prevents duplicate values in a column.
CHECK: Ensures that values satisfy a specified condition. Example: price >= 0.
DEFAULT: Assigns a predefined value when no value is provided. Example: status DEFAULT 'Draft'.

Business Rules & Validation

Cross-table rules require transactional logic beyond simple CHECK constraints.

01	**Traveller capacity**	Confirmed travellers must not exceed departure capacity.
02	**Itinerary sequence**	Day numbers are positive and unique per package.
03	**Guide availability**	A guide cannot be assigned overlapping departures.
04	**Booking confirmation**	Only confirmed departures with available seats may be booked.
05	**Refund validation**	Refund total cannot exceed the eligible payment amount.

IMPLEMENT WITH Constraints · transactions · triggers · stored procedures

SQL Queries, Aggregation & Views

JOINs connect the workflow; aggregation turns transactions into management reports.

CUSTOMER BOOKING DETAILS

```
SELECT c.name, p.package_name, b.status
FROM Booking b
JOIN Customer c ON
c.customer_id=b.customer_id
JOIN Departure d ON
d.departure_id=b.departure_id
JOIN Package p ON
p.package_id=d.package_id;
```

DESTINATION DEMAND

```
SELECT dst.name, COUNT(*) AS
bookings
FROM Booking b
JOIN Departure d ON
d.departure_id=b.departure_id
JOIN Package p ON
p.package_id=d.package_id
JOIN Destination dst ON
dst.destination_id=p.destination_id
GROUP BY dst.name HAVING COUNT(*)
> 0;
```

VIEW: CONFIRMED BOOKINGS

```
CREATE VIEW ConfirmedBookings AS
SELECT b.booking_id, c.name,
d.start_date
FROM Booking b JOIN Customer c
USING(customer_id)
JOIN Departure d USING(departure_id)
WHERE b.status = 'CONFIRMED';
```

Database Prototype & Conclusion

Review 2 establishes a normalized, integrity-first foundation for the final application.

01 Schema + dependencies

02 Data dictionary + DDL

03 Sample data + queries

04 Prototype testing

05 Java / Python integration

TESTING PLAN

PK duplicates · FK validity · capacity · guide conflicts · booking confirmation · cancellation/refund rules · report accuracy

THANK YOU

